Evaluación por prompts: desplazar la evaluación del producto al proceso en la enseñanza de la programación en la era de la IA generativa

Gutierrez Miranda, J. M.

2026-09-03

Los asistentes de programación basados en modelos de lenguaje (ChatGPT, GitHub Copilot, Claude) desacoplan el código entregado del aprendizaje del estudiante: dos alumnos pueden entregar código idéntico habiéndolo comprendido de forma opuesta. Sostengo que este desacoplamiento es un problema de *validez* y no de *integridad*: cuando un generador competente puede producir el artefacto a bajo coste, la información mutua entre el artefacto y la competencia del estudiante tiende a cero, y evaluar el producto —revisar el código del *pull request* línea a línea— pierde poder discriminante además de escalar mal. La respuesta que propongo no es detectar ni prohibir la IA, sino **rediseñar el objeto evaluado**: desplazar la evidencia del producto al proceso. En lugar de código se evalúan los *prompts* que el estudiante dirigió a la IA, las capturas de la interacción y un *blueprint* técnico final, tratados como cuatro fuentes cuya **coherencia mutua** es costosa de falsificar. Enmarco la propuesta en la alineación constructiva y el juicio evaluativo, y describo una implementación abierta —una acción de integración continua que cierra los *pull requests*, una plantilla de documentación y un evaluador heurístico concebido como *clasificador de cribado* orientado a la sensibilidad— desplegada en un curso de Big Data con Python de 230 horas con un grupo heterogéneo de profesionales en activo (perfiles de TI, análisis de datos y big data). La corrección de las entregas pasó de varias horas a decenas de minutos. Discuto con detalle las amenazas a la validez, la evadibilidad de la detección automática y las implicaciones de privacidad. La contribución es metodológica y de práctica —un marco, un método y herramientas reproducibles—, no un estudio controlado de resultados de aprendizaje.

1 Introducción

Desde la difusión masiva de asistentes de programación basados en modelos de lenguaje a finales de 2022, evaluar código como evidencia de aprendizaje presenta un problema de fondo. Herramientas como GitHub Copilot o ChatGPT generan código funcional a partir de una instrucción en lenguaje natural y resuelven una fracción elevada de los ejercicios típicos de un curso introductorio (Finnie-Ansley et al. 2022; Denny et al. 2023). En consecuencia, el artefacto entregado deja de informar de forma fiable sobre lo que el estudiante comprende: dos entregas idénticas pueden corresponder a dos procesos opuestos —una copia sin comprensión y una construcción razonada— y la evaluación no puede distinguirlas mirando solo el resultado.

En la enseñanza de la programación este problema se materializa en el *pull request* (PR), el mecanismo por el que un estudiante propone su trabajo y el docente lo revisa *diff* a *diff*. Revisar un PR busca juzgar la calidad del código; pero si el código puede generarse, esa revisión mide el producto y no el proceso, y además escala mal: decenas de entregas con código similar consumen muchas horas de corrección con escasa información diagnóstica. La comunidad docente ha reaccionado a este escenario en un abanico que va de prohibir las herramientas a integrarlas explícitamente (Lau and Guo 2023; Becker et al. 2023), sin un consenso sobre *qué* evaluar cuando el producto deja de discriminar.

Este trabajo defiende una tesis y documenta su puesta en práctica. La tesis es que el desacoplamiento entre código y aprendizaje es, antes que un problema de integridad académica, un problema de **validez de la evaluación**: el artefacto ha perdido poder discriminante. De ahí que la respuesta más robusta no sea vigilar (detectar copia) ni prohibir (vetar la IA), sino **rediseñar el objeto evaluado** y desplazar la evidencia del *producto* al *proceso* —los *prompts* que el estudiante dirigió a la IA, y las decisiones que tomó con ella—. No se prohíbe la IA: se asume legítima, como herramienta profesional, y se evalúa cómo se la usa.

Las contribuciones son:

1. Un **argumento de validez** (Section 3) que explica, en términos de poder discriminante de la evidencia, por qué el código pierde valor evaluativo ante un generador competente y por qué el proceso lo recupera.
2. Un **método de evaluación por prompts** (Section 4) que trata prompts, capturas, *blueprint* y código como cuatro fuentes cuya **coherencia mutua** es costosa de falsificar, con una rúbrica alineada a los niveles cognitivos superiores.
3. Un **evaluador automático concebido como clasificador de cribado** (Section 4.4), orientado a la sensibilidad (preferir falsos positivos a falsos negativos) para concentrar la revisión humana, con su formulación explícita y sus límites.
4. Un **informe de despliegue** en un curso real (Section 5) y un análisis franco de **amenazas a la validez** (Section 6), privacidad (Section 8) y trabajo futuro, junto con un **conjunto de herramientas abiertas y reproducibles**.

2 Antecedentes y trabajo relacionado

2.1 Evaluación auténtica, alineación y juicio evaluativo

La propuesta se apoya en tres nociones consolidadas de la evaluación educativa. La **evaluación auténtica** sostiene que las tareas deben capturar desempeños significativos y contextualizados, no solo respuestas verificables (Wiggins 1990). La **alineación constructiva** exige que las actividades y los instrumentos de evaluación estén alineados con los resultados de aprendizaje que de verdad se persiguen (Biggs 1996): si el resultado buscado en la era de la IA incluye formular buenos problemas y juzgar críticamente las respuestas de un asistente, evaluar únicamente el código entregado rompe esa alineación. La **evaluación formativa** subraya que el estudiante debe poder comparar su desempeño con un estándar y cerrar la brecha (Sadler 1989), y el desarrollo del **juicio evaluativo** —la capacidad de valorar la calidad del propio trabajo y del ajeno— se propone como competencia central de la educación superior (Tai et al. 2018). La revisión de la **taxonomía de Bloom** (Anderson and Krathwohl 2001) ordena esta transición: cuando *recordar* y *aplicar* sintaxis puede delegarse en la máquina, el nivel cognitivo relevante se desplaza hacia *analizar*, *evaluar* y *crear*, esto es, hacia la formulación de preguntas y la valoración de respuestas.

2.2 IA generativa en la educación en programación

Un cuerpo creciente de literatura documenta el impacto de los generadores de código en la enseñanza. Los modelos resuelven con soltura los exámenes y ejercicios introductorios (Finnie-Ansley et al. 2022), lo que ha llevado a revisar tanto los contenidos como la evaluación de los cursos iniciales (Becker et al. 2023). Frente a ello, el profesorado se posiciona en un espectro que va del veto a la adopción deliberada, con la expectativa mayoritaria de que la resistencia es fútil y de que habrá que rediseñar la evaluación (Lau and Guo 2023). Este trabajo se sitúa en el extremo integrador de ese espectro, pero aporta lo que en esos estudios queda abierto: un *qué* y un *cómo* concretos para evaluar el proceso.

2.3 La formulación de prompts como competencia

Interactuar con un asistente de forma eficaz no es trivial: exige descomponer el problema, expresar restricciones, iterar sobre respuestas parciales y detectar errores. La ingeniería de *prompts* empieza a estudiarse como una competencia con estructura propia —patrones reutilizables (White et al. 2023)— y como objeto de enseñanza en programación introductoria (Denny et al. 2023). Si formular buenos *prompts* es una habilidad que discrimina desempeños, entonces es un candidato natural a *objeto de evaluación*, y no solo un medio para producir el código.

2.4 Integridad y detección: por qué esto no es un detector

Ante la IA generativa, buena parte de la respuesta institucional se ha orientado a la integridad académica y la detección (Cotton et al. 2024). Conviene separar dos linajes. La detección de plagio *entre* entregas tiene una base algorítmica sólida —la huella digital de documentos por *winnowing* es el fundamento de herramientas como MOSS (Schleimer et al. 2003)— y sigue siendo útil frente a la copia literal. Pero la detección de texto *generado por IA* es frágil y evadible, y su uso punitivo traslada al estudiante la carga de la prueba. La propuesta de este trabajo es deliberadamente **distinta de un detector**: no intenta decidir si hubo IA —se asume que la hubo—, sino rediseñar la evidencia evaluada para que el uso honesto y reflexivo de la IA sea, precisamente, lo que produce una buena nota. La detección automática que incluye (Section 4.4) no emite veredictos: solo prioriza qué entregas merecen una conversación con el docente.

3 Por qué el producto pierde validez y el proceso la recupera

Conviene enunciar el argumento con precisión, porque de él se derivan tanto el método como sus límites. Toda evaluación infiere un constructo latente no observable —la competencia del estudiante, C— a partir de evidencia observable E. La calidad de esa inferencia depende del **poder discriminante** de E: de cuánto varía la evidencia con la competencia. En términos informales, una evidencia sirve en la medida en que su información mutua con la competencia, $I(C;E)$, es alta; una evidencia cuya distribución no cambia con C no permite inferir nada.

Antes de los generadores competentes, el código entregado, E_{cod}, era buena evidencia: producir código correcto para un problema no trivial estaba fuertemente condicionado por la competencia, de modo que $I(C;E_{\text{cod}})$ era alta. Un generador capaz cambia ese condicionamiento: el código correcto es alcanzable con esfuerzo y competencia bajos, porque lo aporta la máquina. La distribución del artefacto deja de depender apreciablemente de C y $I(C;E_{\text{cod}}) \to 0$. No es que el estudiante "haga trampa"; es que el artefacto ha dejado de ser informativo. Evaluarlo con la misma rúbrica de antes es un error de medida, con independencia de la honestidad de nadie.

La salida no es restaurar artificialmente el poder discriminante del código —prohibiendo la herramienta o vigilando su uso—, sino **cambiar la evidencia observada** por otra que siga condicionada por la competencia bajo el nuevo régimen. El proceso —los *prompts*, las iteraciones, las decisiones, la capacidad de explicar lo construido— cumple esa condición: el generador produce el código, pero no produce *por el estudiante* la secuencia de preguntas informadas que lleva a un buen resultado ni la comprensión que permite defenderlo. Sea E_{proc} esa evidencia de proceso; la apuesta del método es que, en el régimen actual, $I(C;E_{\text{proc}}) \gg I(C;E_{\text{cod}})$. Esto es, exactamente, restaurar la **alineación constructiva** (Biggs 1996): alinear la evidencia evaluada con el resultado de aprendizaje que hoy importa —formular y juzgar—, en lugar de con un artefacto que la máquina ya provee.

4 El método: evaluación por prompts

4.1 Modelo de evidencia: cuatro fuentes coherentes

El entregable principal deja de ser el código y pasa a ser un archivo `PROMPTS.md` acompañado de material de apoyo. Se evalúan cuatro fuentes:

1. **Prompts reales**, copiados tal cual —con sus erratas e informalidad—, que muestran cómo el estudiante descompuso el problema e interrogó a la IA.
2. **Capturas** de las interacciones clave, como evidencia de que los *prompts* ocurrieron y en qué contexto.
3. Un **blueprint** técnico final que la propia IA redacta a petición del estudiante —*stack*, arquitectura, decisiones— y que obliga a articular lo construido.
4. El **código** resultante, ya no como objeto central de la nota sino como ancla de coherencia.

La fuerza del método no reside en ninguna fuente aislada —cada una es falsificable por separado— sino en su **coherencia mutua**. Los *prompts* deben explicar el código; las capturas deben corresponderse con los *prompts*; el *blueprint* debe describir lo que el código realmente hace. Fabricar un caso coherente en las cuatro fuentes a la vez —*prompts* verosímiles con su rastro humano, capturas que coinciden, un *blueprint* fiel a un código que el estudiante no entendió— tiene un coste que se aproxima al de haber hecho el trabajo. El método no vuelve imposible el engaño: eleva su coste hasta hacerlo comparable al del aprendizaje, que es el objetivo razonable de un diseño de evaluación.

4.2 Rúbrica alineada a los niveles cognitivos superiores

La valoración humana se guía por una rúbrica que premia lo que discrimina competencia bajo el nuevo régimen: la **calidad de la descomposición** del problema en *prompts*; la **capacidad de iterar** ante respuestas erróneas (detectar el fallo, reformular); la **coherencia** entre proceso, *blueprint* y código; y la **comprensión** demostrada al explicar decisiones. Estos criterios corresponden a *analizar*, *evaluar* y *crear* (Anderson and Krathwohl 2001), y son observables en el proceso aunque el código lo genere la máquina. La ingeniería de *prompts* aporta, además, un vocabulario de patrones con el que describir y calificar esa descomposición (White et al. 2023).

4.3 Cierre del canal de pull requests

Una acción de integración continua detecta la apertura de cualquier PR, publica un comentario que explica el sistema y lo cierra. El canal de aporte deja de ser el *diff* —cuya revisión ya no discrimina— y pasa a ser la propuesta documentada. El cierre no es un veto tecnológico caprichoso: es la señal, coherente con el diseño, de que el objeto evaluado ha cambiado.

4.4 El evaluador como clasificador de cribado

Corregir a mano decenas de entregas de proceso volvería a ser costoso. El sistema incluye un evaluador que recorre las entregas y produce un informe con indicadores objetivos —presencia y número de *prompts*, existencia de capturas y de *blueprint*, extensión— y una nota orientativa. Su papel no es calificar sino **cribar**: es un clasificador deliberadamente orientado a la **sensibilidad**, que prefiere marcar de más (falsos positivos, entregas correctas enviadas a revisión) antes que de menos (falsos negativos, entregas problemáticas que pasan inadvertidas). El docente confirma; la máquina solo prioriza.

La nota orientativa sobre 100 combina, de forma transparente y editable, número de *prompts* (hasta 30), presencia de capturas (+20) y de *blueprint* (+20) y extensión como aproximación al esfuerzo (hasta 30), con ajustes por alertas: *prompts* "demasiado limpios" —sin el rastro humano de erratas e informalidad—, contenido escaso o incoherencia con el ejemplo del docente. Estas señales son heurísticas y **evadibles** por diseño (véase Section 6): su función es ordenar la cola de revisión, no sustituir el juicio del docente. La categorización resultante (revisar / normal / destacado / rechazado) determina únicamente **dónde mira primero** el evaluador humano.

5 Despliegue e implementación

El sistema se aplicó en un curso de **Big Data con Python** de 230 horas cuyo grupo no era de estudiantes universitarios convencionales, sino un conjunto **heterogéneo de profesionales en activo** —perfiles previos en TI, análisis de datos y big data— que cursaban el programa por necesidad de actualización profesional. Esta composición es relevante para interpretar los resultados: se trata de adultos con experiencia técnica y motivación instrumental, no de principiantes en un primer contacto con la programación.

La corrección de las entregas finales, que con revisión de PRs requería varias horas, se realizó en el orden de media hora: el evaluador de cribado ordenó la cola y el docente intervino a mano solo en los casos marcados. Este resultado se reporta como **experiencia de despliegue**, no como medida de un estudio controlado: no se compararon resultados de aprendizaje frente a un grupo de control, ni se estimó la fiabilidad entre evaluadores ni la tasa de falsos positivos del cribado (véase Section 6). El código, las plantillas y el evaluador se publican para que otros docentes los adapten a sus propios materiales.

6 Amenazas a la validez

Validez de constructo. El método asume que la calidad del proceso documentado mide competencia. Un estudiante hábil con la IA pero con comprensión superficial podría obtener buena nota; a la inversa, un buen programador parco documentando podría salir perjudicado.

La rúbrica mitiga lo primero exigiendo coherencia y capacidad de explicar decisiones, y lo segundo tratando la documentación como competencia profesional legítima —comunicar el porqué es parte del oficio (Hunt and Thomas 2019)—, pero la correspondencia entre proceso y comprensión debe validarse empíricamente frente a medidas independientes.

Validez interna: juego adversario y falsos positivos. La detección de *prompts* fabricados es heurística y **evadible**: basta pedir a la IA que introduzca erratas para simular espontaneidad, o reconstruir *a posteriori* un rastro verosímil. Por eso el cribado no emite veredictos. Simétricamente, el sesgo hacia la sensibilidad genera falsos positivos: estudiantes de estilo formal pueden activar la alerta sin haber copiado, lo que hace imprescindible que la señal conduzca a una conversación y no a una sanción. La coherencia entre cuatro fuentes eleva el coste del juego, pero no lo anula.

Validez externa. La evidencia procede de **un único despliegue**, en **un solo curso**, con un **grupo de profesionales adultos** de un dominio concreto (big data). La generalización a estudiantes de grado sin experiencia previa, a otros dominios o a cursos introductorios es una conjetura, no un resultado: cabe esperar que en principiantes la frontera entre "usar la IA para aprender" y "usarla para eludir el aprendizaje" sea más difusa y el diseño de la rúbrica más delicado.

Validez de conclusión. La reducción del tiempo de corrección (de horas a decenas de minutos) es una observación de eficiencia, no una medida controlada, y depende del tamaño del grupo y de la práctica del docente. No debe leerse como una estimación con incertidumbre acotada.

7 Alcance: cohorte conocida frente a aporte externo

El método reduce el tiempo de revisión y aumenta la productividad del docente, pero descansa sobre un supuesto que conviene explicitar. Dentro de un curso o de un equipo de desarrollo, quien evalúa **conoce a su gente**: reconoce la voz de un estudiante en sus *prompts* igual que reconocería su manera de escribir código, y la coherencia entre el proceso y la persona le resulta verificable casi de un vistazo. En ese régimen, la automatización descrita —el cribado que ordena la cola de revisión— libera tiempo precisamente porque el juicio de fondo se apoya en un conocimiento previo del evaluado.

La naturaleza del problema cambia cuando la propuesta llega de **alguien a quien no se conoce**. Es el caso del aporte externo a un proyecto público —de código abierto—, que es, no por casualidad, el escenario para el que se concibieron los *pull requests*. Allí no basta con leer los *prompts*: hay que leer *entre líneas* cómo se construyó el razonamiento, **probar** el aporte y decidir si se acepta o se rechaza aplicando la experiencia y el conocimiento del oficio. Esa decisión conserva una **subjetividad irreducible**: no puede automatizarse, ni preverse, ni erradicarse, y es exactamente la parte que, como humanos, seguimos teniendo que gestionar. Conviene, por tanto, delimitar el alcance de la propuesta: la evaluación por *prompts* desplaza y reduce el trabajo de revisión dentro de una relación de confianza ya establecida —el

aula, el equipo—, pero **no elimina el juicio humano** allí donde esa confianza no existe de antemano. Reconocer este límite no debilita el método; lo sitúa: automatiza el cribado, no la responsabilidad de decidir.

8 Ética y privacidad

El método pide a los estudiantes publicar *prompts* y capturas en un repositorio público, lo que exige cautelas explícitas. Debe obtenerse **consentimiento informado** y ofrecerse una **alternativa privada** —repositorio privado o entrega por el aula virtual— para quien no desee exponer su trabajo, sin penalización. Las capturas pueden contener datos personales (nombre, correo, interfaz de la cuenta), por lo que se instruye a los estudiantes para que los **oculten** antes de subir. La aplicación debe cumplir la normativa de protección de datos de la institución (RGPD en la Unión Europea, FERPA en Estados Unidos), pseudonimizar cuando sea posible y acordar la retención y la retirada de los trabajos a petición del estudiante. Estas cautelas no son accesorias: un método que gana transparencia sobre el proceso incrementa por ello mismo la exposición del estudiante, y ese equilibrio debe gestionarse de forma expresa.

9 Trabajo futuro

La agenda empírica que este trabajo deja abierta es concreta: (i) **validar la rúbrica** correlacionando la evaluación por *prompts* con medidas independientes de comprensión (p. ej. una defensa oral o un examen); (ii) estimar la **fiabilidad entre evaluadores** de la rúbrica; (iii) medir la **tasa de falsos positivos** del cribado y calibrar su umbral de sensibilidad; (iv) replicar en **varios cursos e instituciones**, incluyendo grados con principiantes, para acotar la validez externa; y (v) explorar la **transferencia** del principio —evaluar el proceso de pensamiento en lugar del entregable que la IA puede generar— a dominios no relacionados con la programación, donde cambia el formato de la evidencia pero no la lógica del argumento.

10 Conclusión

Cuando un generador competente puede producir el artefacto, evaluar el producto deja de medir el aprendizaje: es un problema de validez antes que de honestidad. La respuesta más robusta no es prohibir ni vigilar la IA, sino rediseñar el objeto evaluado y desplazar la evidencia al proceso —los *prompts*, las decisiones, la capacidad de explicar—, restaurando la alineación entre lo que se evalúa y lo que hoy se aprende. La evaluación por *prompts* operacionaliza ese desplazamiento con un modelo de evidencia basado en la coherencia de varias fuentes, una rúbrica orientada a los niveles cognitivos superiores y un cribado automático que prioriza la revisión humana sin sustituirla. Su alcance tiene además un límite honesto (Section 7): automatiza el cribado dentro de una relación de confianza conocida —el aula, el equipo—,

pero no el juicio, irreduciblemente subjetivo, que exige aceptar o rechazar el aporte de un desconocido. Se ofrece como marco, método y herramientas reproducibles, a la espera de la evidencia empírica que un estudio controlado deberá aportar.

Disponibilidad

El código, las plantillas y el evaluador están disponibles bajo licencia MIT en https://github.com/TodoEconometria/evaluacion-prompts-ia.

Cómo citar este trabajo

Gutierrez Miranda, J. M. (2026). *Evaluación por prompts: desplazar la evaluación del producto al proceso en la enseñanza de la programación en la era de la IA generativa.* Preprint. TodoEconometria.

Referencias

Anderson, Lorin W., and David R. Krathwohl, eds. 2001. *A Taxonomy for Learning, Teaching, and Assessing: A Revision of Bloom's Taxonomy of Educational Objectives*. Longman.

Becker, Brett A., Paul Denny, James Finnie-Ansley, Andrew Luxton-Reilly, James Prather, and Eddie Antonio Santos. 2023. "Programming Is Hard — or at Least It Used to Be: Educational Opportunities and Challenges of AI Code Generation." *Proceedings of the 54th ACM Technical Symposium on Computer Science Education (SIGCSE 2023), Volume 1*, 500–506. https://doi.org/10.1145/3545945.3569759.

Biggs, John. 1996. "Enhancing Teaching Through Constructive Alignment." *Higher Education* 32 (3): 347–64. https://doi.org/10.1007/BF00138871.

Cotton, Debby R. E., Peter A. Cotton, and J. Reuben Shipway. 2024. "Chatting and Cheating: Ensuring Academic Integrity in the Era of ChatGPT." *Innovations in Education and Teaching International* 61 (2): 228–39. https://doi.org/10.1080/14703297.2023.2190148.

Denny, Paul, Viraj Kumar, and Nasser Giacaman. 2023. "Conversing with Copilot: Exploring Prompt Engineering for Solving CS1 Problems Using Natural Language." *Proceedings of the 54th ACM Technical Symposium on Computer Science Education (SIGCSE 2023), Volume 1*, 1136–42. https://doi.org/10.1145/3545945.3569823.

Finnie-Ansley, James, Paul Denny, Brett A. Becker, Andrew Luxton-Reilly, and James Prather. 2022. "The Robots Are Coming: Exploring the Implications of OpenAI Codex on Introductory Programming." *Proceedings of the 24th Australasian Computing Education Conference (ACE '22)*, 10–19. https://doi.org/10.1145/3511861.3511863.

Hunt, Andrew, and David Thomas. 2019. *The Pragmatic Programmer: Your Journey to Mastery*. 20th Anniversary. Addison-Wesley.

Lau, Sam, and Philip J. Guo. 2023. "From 'Ban It till We Understand It' to 'Resistance Is Futile': How University Programming Instructors Plan to Adapt as More Students Use AI Code Generation and Explanation Tools Such as ChatGPT and GitHub Copilot." *Proceedings of the 2023 ACM Conference on International Computing Education Research (ICER '23), Volume 1*, 106–21. https://doi.org/10.1145/3568813.3600138.

Sadler, D. Royce. 1989. "Formative Assessment and the Design of Instructional Systems." *Instructional Science* 18 (2): 119–44.

Schleimer, Saul, Daniel S. Wilkerson, and Alex Aiken. 2003. "Winnowing: Local Algorithms for Document Fingerprinting." *Proceedings of the 2003 ACM SIGMOD International Conference on Management of Data*, 76–85. https://doi.org/10.1145/872757.872770.

Tai, Joanna, Rola Ajjawi, David Boud, Phillip Dawson, and Ernesto Panadero. 2018. "Developing Evaluative Judgement in Higher Education: Assessment for Knowing and Producing Quality Work." *Higher Education* 76 (3): 467–81. https://doi.org/10.1007/s10734-017-0220-3.

White, Jules, Quchen Fu, Sam Hays, et al. 2023. "A Prompt Pattern Catalog to Enhance Prompt Engineering with ChatGPT." *arXiv Preprint arXiv:2302.11382*, ahead of print. https://doi.org/10.48550/arXiv.2302.11382.

Wiggins, Grant. 1990. "The Case for Authentic Assessment." *Practical Assessment, Research, and Evaluation* 2 (2).